AgentInbox

Configure two mailboxes from your terminal

I wanted one assistant connection for all my mailboxes. I built AgentInbox so you can configure existing Gmail, Microsoft, iCloud, GoDaddy and other compatible mailboxes locally, then expose the accounts and folders you choose through one MCP connection.

This guide connects one Gmail account and one Microsoft 365 work or school account. Both start with IMAP, INBOX and reading enabled. Sending starts disabled. The addresses ending in `example.test` are invented; replace them with your existing mailboxes. AgentInbox does not create accounts at Google or Microsoft.

1. Choose the profile and setup-file locations

Use Python 3.11 or newer. Run the commands from the AgentInbox source checkout. If your system names the interpreter `python3`, use that name throughout. The setup file goes at `~/AgentInbox/accounts.setup.json`; AgentInbox writes its managed data under `~/AgentInbox/profile`.

PowerShell, on Windows:

```
New-Item -ItemType Directory -Force "$HOME/AgentInbox"
$env:AGENTINBOX_DATA_DIR = "$HOME/AgentInbox/profile"
$setup = "$HOME/AgentInbox/accounts.setup.json"
Copy-Item docs/examples/accounts.setup.json $setup
python -B python/run.py profile show
```

Bash or zsh, on Linux/macOS:

```
mkdir -p "$HOME/AgentInbox"
export AGENTINBOX_DATA_DIR="$HOME/AgentInbox/profile"
setup="$HOME/AgentInbox/accounts.setup.json"
cp docs/examples/accounts.setup.json "$setup"
python -B python/run.py profile show
```

Copy the template once. Edit that copy before adding accounts. Do not overwrite an existing edited file. The variable assignments apply to this terminal; set the same absolute profile path in each new terminal and in your MCP client's environment.

Windows uses Credential Manager. macOS needs an accessible Keychain. Linux needs libsecret and an unlocked Secret Service keyring in the user session running the process. Linux/macOS execution remains unverified. These new CLI commands are source changes and are not in the currently installed MSIX.

2. Prepare Gmail

Use your existing Gmail address in both `email` and `incoming.username`. The example labels it `gmail.reader@example.test` so nobody accidentally treats it as a real account.

1. Open Google Account security and enable 2-Step Verification.
2. Open App passwords, create a password for AgentInbox and keep it ready for the terminal prompt.
3. When `accounts add` asks for the Gmail app password, enter that generated value using hidden input. Do not enter your ordinary Google password, device password or PIN. Do not put it in the JSON file or shell command.

App passwords are available only for eligible accounts. Organization policies, security-key-only settings or Advanced Protection can prevent the option from appearing. Ask your administrator if your managed account does not allow it. See Google's app-password instructions.

For personal Gmail, IMAP is always enabled; the old Enable IMAP switch is no longer required. Google Workspace policy can differ. See Gmail client access.

What AgentInbox stores

- Gmail: the app password is persisted in the operating system's protected credential store. It is not written into the setup JSON or the managed settings JSON.
- Microsoft: your Microsoft password is entered only on Microsoft's sign-in page. AgentInbox neither requests nor saves that password.
- Microsoft OAuth: AgentInbox does persist a refresh token in the protected credential store so it can renew access. Short-lived access tokens stay in process memory. Temporary sign-in state is also protected.

The two JSON files have different jobs

`~/AgentInbox/accounts.setup.json` is your editable settings template. It contains the two entries shown later in this guide. Adding an account does not rewrite this file.

`~/AgentInbox/profile/accounts.json` is managed by AgentInbox. It contains saved settings, generated IDs, revisions and credential references. Never paste the setup example over this file or edit credential references by hand. Use `accounts add` and `accounts update` instead.

3. Prepare Microsoft 365

The `microsoft` example is for a work or school mailbox in Exchange Online. Replace its invented address and the public client-ID placeholder before adding it.

Enable the mailbox protocol

An administrator opens the Exchange admin center, then goes to Recipients > Mailboxes and selects the mailbox. Under General > Email apps & mobile devices, open Manage email apps settings. Set IMAP to Enabled and save. Enable POP3 only if you actually choose POP3 instead of IMAP. These are mailbox-access settings, separate from OAuth consent. Microsoft's mailbox access instructions.

The Microsoft 365 admin center is the general administration portal; the Exchange portal above controls these mailbox settings. Organization policy may require an administrator to approve access.

Supply a public-client registration

The source CLI needs the public application client ID in the JSON. Use a registration provided by the maintainer or one your organization has configured for this purpose. Windows graphical setup supplies its shipped ID separately.

In the Microsoft Entra admin center, open the application's registration. Confirm its supported account types match the mailbox. Under Authentication > Advanced settings, set Allow public client flows to Yes and save. Copy the Application (client) ID into `authentication.client_id`. A client ID is not a password; do not create or place a client secret in this file. Microsoft public-client setup.

For IMAP, the app requests the delegated Exchange Online scope `https://outlook.office.com/IMAP.AccessAsUser.All` plus `offline_access`. Configure the required delegated permission and obtain user/admin consent as your tenant requires. POP3 uses `POP.AccessAsUser.All`; enabled sending also needs `SMTP.Send`. These are Exchange mail protocol permissions, not Microsoft Graph Mail.Read. Microsoft OAuth protocol guide.

For personal Outlook.com, use `preset: "outlook"` and `tenant: "consumers"` with a registration supporting personal accounts. Enable IMAP in the mailbox's own settings rather than an organization's admin portal. Outlook.com settings.

4. The complete two-account JSON

Save this as `~/AgentInbox/accounts.setup.json`. Replace both invented addresses, including their username fields, and `REPLACE_WITH_PUBLIC_CLIENT_ID`. Keep each `id` empty. The example is valid JSON but the Microsoft placeholder is deliberately rejected until you replace it with a real public client GUID.

Editable example (included with this guide)

```
{
  "accounts": {
    "gmail": {
      "id": "",
      "label": "My Gmail mailbox",
      "email": "gmail.reader@example.test",
      "preset": "gmail",
      "incoming": {
        "protocol": "imap",
        "host": "imap.gmail.com",
        "port": 993,
        "tls": "tls",
        "username": "gmail.reader@example.test"
      },
      "smtp": null,
      "authentication": { "method": "password" },
      "read_enabled": true,
      "send_enabled": false,
      "folders": ["INBOX"]
    },
    "microsoft": {
      "id": "",
      "label": "My Microsoft 365 mailbox",
      "email": "microsoft.reader@example.test",
      "preset": "microsoft365",
      "incoming": {
        "protocol": "imap",
        "host": "outlook.office365.com",
        "port": 993,
        "tls": "tls",
        "username": "microsoft.reader@example.test"
      },
      "smtp": null,
      "authentication": {
        "method": "microsoft_oauth",
        "client_id": "REPLACE_WITH_PUBLIC_CLIENT_ID",
        "tenant": "organizations"
      },
      "read_enabled": true,
      "send_enabled": false,
      "folders": ["INBOX"]
    }
  }
}
```

No password, access token or refresh token belongs in this file. Each command selects one named entry; it does not add both accounts at once.

5. Add, sign in and check

Keep the `setup` variable and `AGENTINBOX_DATA_DIR` from page 1. Run these commands from the source checkout. Replace `GMAIL_ID` and `MICROSOFT_ID` with the IDs printed by the add commands.

Add Gmail

```
python -B python/run.py accounts add --file "$setup" --entry gmail
```

The terminal asks for the Gmail app password without showing what you type. A saved-account result confirms local configuration only. Record its ID, then explicitly check the provider connection:

```
python -B python/run.py accounts check GMAIL_ID
```

Add Microsoft and authorize it

```
python -B python/run.py accounts add --file "$setup" --entry microsoft
python -B python/run.py accounts signin MICROSOFT_ID
```

The sign-in command prints a secure Microsoft URL and a short code. Open the URL yourself, enter the code and sign in to the intended account. The terminal waits at the interval requested by Microsoft. Ctrl+C stops the local flow. AgentInbox never asks you to type a Microsoft password into the terminal.

After sign-in succeeds, check mail access separately:

```
python -B python/run.py accounts check MICROSOFT_ID
python -B python/run.py accounts list
python -B python/run.py accounts folders MICROSOFT_ID
```

Checks authenticate and inspect configured connections. They do not send email. A sample read happens only with `accounts check ID --read-message`; it returns the outcome without message content. Repeating `accounts add` creates another connection. Use `accounts update` for an existing one.

Use one MCP connection

Configure your MCP client to start an absolute Python executable with separate arguments `-B`, the absolute path to `python/run.py`, and `mcp`. Set its `AGENTINBOX_DATA_DIR` to the same absolute profile directory. Both enabled accounts are then exposed through that process.

Starting MCP or saving an account does not prove a mailbox works. The new CLI flows, provider access and cross-platform behavior still require runtime verification. The PDF's visual review is not an application test.

6. Maintain your configuration

Inspect the current revision before updating a connection. Edit the same named entry in your setup file, keep its `id` empty, and replace `ID` and `N` below with the saved account ID and revision:

```
python -B python/run.py accounts show ID
python -B python/run.py accounts update ID --revision N --file "$setup" --entry gmail
```

For a password replacement or changed password-based server settings, add `--replace-passwords`. The CLI asks for incoming and configured SMTP passwords separately; enter the app password again if both use it. Stale revisions fail before saving. Use `--entry microsoft` for the Microsoft entry and sign in again after changes that require new consent.

Optional sending

The example uses `smtp: null` and `send_enabled: false`. To enable sending deliberately, set `send_enabled` to true and replace `smtp` with the appropriate object, using your actual email address as the username:

Gmail SMTP:

```
{"host":"smtp.gmail.com","port":465,"tls":"tls",
 "username":"YOUR_GMAIL_ADDRESS"}
```

Microsoft 365 SMTP:

```
{"host":"smtp.office365.com","port":587,"tls":"starttls",
 "username":"YOUR_MICROSOFT_ADDRESS"}
```

Update Gmail with `--replace-passwords`. Update Microsoft, then run `accounts signin ID` again to request sending permission. Your administrator may also need to allow Authenticated SMTP for that mailbox under the tenant's policies. Microsoft SMTP AUTH settings. Check with `accounts check-smtp ID`; it sends no message. Actual sending remains an explicitly requested MCP operation.

Useful commands

```
python -B python/run.py accounts list --json
python -B python/run.py profile show
python -B python/run.py profile cleanup
python -B python/run.py accounts remove ID
python -B python/run.py accounts --help
```

Removal deletes the local connection and its credentials, not the provider mailbox. Revoke provider access separately if you no longer need it. A cleanup warning does not undo a saved account or successful sign-in: retry cleanup, not the original action. `--json` gives automation one structured result; prompts and progress stay on stderr. Advanced integrations can continue using `manage` JSON requests.

Prepared on 26 October 2026 from the current source and the official links in this guide. No live mailbox or sign-in was used. I'm Umberto Giacobbi. I built and maintain AgentInbox, with help from contributors.